

LLM-DoW-Bench: A Benchmark and Threat Model for Denial-of-Wallet Attacks Against LLM Tool-Use Agents

Anonymous Author(s)
Submitted to IEEE SaTML 2027

Abstract—LLM agents with tool access can be induced, via indirect prompt injection or adversarial task framing, into consuming far more cost than a task warrants – without any individually unauthorized action. We term this a *Denial-of-Wallet (DoW)* attack: the victim’s systems remain available; only their bill is affected. Because every action is individually permitted, action-legitimacy defenses do not obviously transfer, and existing agent-security benchmarks measure only behavioral compromise, never cost. We present LLM-DoW-Bench: a taxonomy of six cost-amplification vectors spanning two agentic task domains, and a LangGraph-based cost-metering harness. We evaluate six frontier models across five trials per condition for five of the six vectors; the sixth (adversarial task framing, no injection) is included in the taxonomy but not yet validly instrumented, which we report as a negative methodological result. Cost-amplification is sharply heterogeneous: context/input bloat amplifies cost severely for every model (mean $6.4\times$, up to $21.5\times$), while retry induction and expensive-tool bait are weak (mean $1.2\text{--}1.3\times$). The two delegation vectors show a per-model split: four of six models resist injected delegation instructions almost perfectly, while two OpenAI models comply substantially. We evaluate two of four candidate defenses: a hard cumulative-cost budget and an intent-consistency LLM judge. Contrary to prediction, the budget is nearly inert (0.7% attack-block rate); our pre-tool-call gate cannot intervene on cost accrued within a single call, the mechanism the dominant vector exploits. The intent judge, predicted to show less resistance, instead blocks more attack trials (13.3%), but only for the delegation vectors and models already susceptible to them, at a small false-positive cost. Both defenses leave the dominant vector untouched.

Index Terms—LLM agents, prompt injection, denial of wallet, resource exhaustion, economic denial of sustainability, agent security benchmarks

I. INTRODUCTION

LLM agents with tool access – shell execution, cloud APIs, database queries, paid third-party APIs, sub-agent spawning – are increasingly deployed to complete multi-step tasks with limited human supervision. This autonomy creates an attack surface distinct from the one usually discussed for prompt injection. The standard framing of indirect prompt injection concerns an agent tricked, via untrusted content it reads, into taking an unauthorized or harmful action: exfiltrating data, executing a destructive command, or violating an access boundary. That framing assumes the harm is legible as a rule violation – some action the agent took was not permitted.

A different failure mode does not require any rule violation at all. An agent can be induced, by the same delivery mechanisms (a poisoned tool result, a page it is asked to

summarize, an adversarially framed task), into doing something entirely permitted – searching, calling an API, delegating to a sub-agent – an excessive number of times, or in an unnecessarily expensive way. No confidentiality or integrity boundary is crossed. The agent’s owner simply pays for work the task did not need. We refer to this as a Denial-of-Wallet (DoW) attack against LLM tool-use agents, adopting a term with an established academic lineage in serverless-computing security [1]–[3] and separately reapplied, informally, in generative-AI industry security writing [4]. Two concurrent 2026 papers, Zhou et al. [5] and SkillBloat [6], independently measure cost/token amplification through an agent-adjacent channel, but under threat models and with a scope this paper’s contribution is specifically positioned against (Section II). A recent systematization of knowledge on the agentic-AI attack surface, Dehghantanha et al. [7], surveys prompt-injection, knowledge-base-poisoning, tool/plugin-exploit, and multi-agent threats broadly and proposes its own evaluation metrics (Unsafe Action Rate, Privilege Escalation Distance), but does not address cost- or resource-exhaustion attacks, and its proposed metrics target unsafe or privileged actions rather than economic cost – underscoring that even a broad, recent systematization of the agentic-AI threat landscape does not yet cover the failure mode this paper studies. To our knowledge, no existing work combines an indirect-prompt-injection-style threat model, a taxonomy spanning both single-agent and multi-agent/delegation vectors, and a reusable, cost-instrumented benchmark with a paired defense evaluation, for the agentic tool-use setting.

This failure mode has a non-adversarial counterpart: OWASP’s LLM Top 10 explicitly names “Unbounded Consumption” as a production risk for LLM deployments, noting that unintentional retry loops, unmonitored tool invocations, and uncapped agent fan-out create the same cost exposure that our adversarial corpus targets [8]. Our interest is the adversarial version of that same exposure: an attacker who deliberately engineers a task, document, or tool response to maximize an agent’s cost.

Why existing defenses don’t obviously transfer. Intent-consistency checks and permission-scoping mechanisms are built to catch an action that should not have been taken at all. A DoW attack does not require such an action – every individual tool call can be exactly the kind of call the agent’s permissions were designed to allow. This is the cost-specific instance of

a more general point independently argued by Lotfi et al. [9] for regulatory/behavioral-compliance invariants: a sequence of individually permitted agent actions can collectively violate a trajectory-level invariant that no single-action check can see. Section VI runs an actual defense evaluation against this prediction, and the result is more mixed than the clean hypothesis above: an intent-consistency judge does show some attack-resistance (not none), while a budget-aware hard cap shows almost no resistance at all, for a structural reason (Section VI) rather than because budgets are inherently weak.

Why existing benchmarks don’t measure this. We reviewed four widely cited, methodologically rigorous agent-security benchmarks – AgentDojo [10], InjecAgent [11], ToolEmu [12], and R-Judge [13] – and confirmed that none of them instrument for token count, tool-call count, dollar cost, wall-clock time, or sub-agent fan-out as an evaluation outcome. Their outcome variable is uniformly behavioral compromise (an unauthorized or harmful action) or data exfiltration, not economic exhaustion. Two concurrent 2026 works do measure cost/token amplification directly – Zhou et al. [5] via a malicious-MCP-server threat model on single-tool tasks, and SkillBloat [6] via malicious skill-file injection in coding agents – but neither evaluates multi-agent delegation, and neither releases a reusable, multi-domain benchmark artifact of the kind this paper contributes (Section II).

Contributions.

- A threat model for DoW attacks against LLM tool-use agents specifically, distinguishing this attack surface from (a) classical resource-exhaustion / Economic Denial of Sustainability (EDoS) attacks on non-agentic cloud infrastructure [14], (b) existing LLM-level denial-of-service attacks that target the base model or serving stack directly rather than the tool-use/orchestration layer [15]–[18], and (c) concurrent tool-layer cost-amplification work that assumes a stronger (server-compromise or single-product-feature) adversary and does not cover multi-agent delegation [5], [6] (Section III).
- LLM-DoW-Bench: a reproducible benchmark comprising an attack corpus of six vectors across two agentic task domains and a LangGraph-based cost-metering harness (Section IV).
- An empirical evaluation of six frontier tool-use agent configurations’ susceptibility to DoW attacks across five trials per condition, measured by cost-amplification factor for five of the six taxonomy vectors (V1–V4, V6), finding sharp vector- and model-dependent heterogeneity rather than a uniform effect; the sixth vector’s (V5) corpus entry is diagnosed as not yet validly instrumented, which we report as a methodological finding in its own right rather than omit (Section V).
- A defense evaluation (Section VI) of two candidate defenses – a hard cumulative-cost budget and an intent-consistency LLM-judge – against this corpus, finding that neither provides the clean, uniform “meaningful resistance” our own threat model predicted for budget-aware mechanisms, and that a pre-tool-call gate architecture

(the natural implementation for both defense classes) is structurally unable to intervene on the specific vector (V4, context/input bloat) that causes the most severe cost amplification in Section V.

II. RELATED WORK

A. Indirect Prompt Injection and Agent-Security Benchmarks

Indirect prompt injection – an agent’s behavior redirected by adversarial content embedded in data it processes, rather than by an instruction from its principal – is well established as an attack class. AgentDojo [10] evaluates 97 realistic agent tasks against 629 security test cases across email, banking, and travel-booking-style environments; InjecAgent [11] evaluates 1,054 test cases across 30 agents, categorizing outcomes into direct user harm and private data exfiltration; ToolEmu [12] uses an LM-emulated sandbox across 36 tools and 144 scenarios to surface safety failures; R-Judge [13] evaluates whether models can recognize safety risk in 569 multi-turn agent interaction records across 10 risk types. The largest and most recent benchmark in this space, Agent Security Bench (ASB) [19], spans 10 scenarios, 10 agents, over 400 tools, and 27 attack/defense types across 7 evaluation metrics. We verified, by reviewing each benchmark’s methodology directly, that none of these five measures token count, tool-call count, dollar cost, or fan-out depth as an outcome variable – ASB’s metrics center on attack success rate (up to 84.3%) and a utility-security balance score, not economic exhaustion. All five treat the outcome of an attack as a behavioral or data-integrity compromise, not an economic one. A sixth, concurrent 2026 benchmark, AgentLAB [20], evaluates 644 test cases across 28 environments against five long-horizon attack categories, one of which – tool chaining – mechanistically overlaps with this paper’s delegation vectors (V2, V6); however, AgentLAB’s outcome metric is attack success rate, not cost or token amplification, so it does not close the gap this paper targets. This project targets that specific gap: an equally rigorous benchmark whose outcome variable is cost.

B. Cost-Amplification Attacks Through the Tool-Use Layer (Concurrent Work)

Two concurrent 2026 papers attack cost through the tool-calling layer specifically, the same layer this paper targets, and are the closest existing work to our contribution. Zhou et al. [5] assume an adversary who controls a malicious MCP tool *server* and rewrites its response templates – a supply-chain-adjacent capability, not an indirect-prompt-injection one – and use Monte Carlo Tree Search to synthesize multi-turn tool-calling chains that inflate cost by up to 658× on single-tool, single-turn subsets of ToolBench and BFCL; they evaluate several defenses (perplexity filters, self-reflection monitoring, trajectory-level safety judges, hard budget caps), all evaded or only partially effective, but release an attack-generation method rather than a reusable benchmark artifact, and do not address multi-agent delegation. SkillBloat [6] targets a narrower channel – the “Agent Skills” feature of coding agents specifically – using an automated two-phase framework to

rewrite a skill document for maximum token amplification, reporting $5.4\times$ – $10.1\times$ average best amplification; it likewise does not address multi-agent delegation or evaluate defenses. Both works confirm that cost/token amplification through an agent-adjacent channel is measurable and severe, which motivates rather than undercuts this paper’s contribution. A third concurrent work, LeechHijack [21], hijacks agent compute for attacker-specified tasks via a backdoored Model Context Protocol (MCP) tool exhibiting “implicit toxicity” – exploiting granted permissions without privilege escalation – reporting an 18.6% token-cost overhead and 77.3% average attack success across model families; like Zhou et al., it assumes a compromised tool rather than merely poisoned content an otherwise-trustworthy tool returns, and releases no reusable benchmark artifact. For the multi-agent setting specifically, CORBA [22] is the closest identified prior work to our delegation-loop vector (V6): a “contagious” attack that propagates across arbitrary multi-agent network topologies via ordinary message-passing (no delegation-tool primitive required) and a “recursive” self-loop property that sustains computational-resource depletion via seemingly benign instructions. We read CORBA in full and confirm it measures a fundamentally different asset than ours: its outcome metrics are Proportional Attack Success Rate (the fraction of agents driven to stop responding, up to 100%) and Peak Blocking Turn Number – an availability/blocking metric, not an economic one – and it reports no cost, token, or dollar-amplification ratio. V6 measures whether a functioning agent’s *cost* is amplified by unbounded delegation depth; CORBA measures whether agents stop responding at all. We treat this as complementary rather than competing: CORBA and this paper study different STRIDE-adjacent harms (availability vs. budget) arising from structurally similar multi-agent propagation dynamics.

Our threat model differs from all four in three respects: (1) we assume only the standard indirect-prompt-injection capability already used by AgentDojo, InjecAgent, and ToolEmu (C1: poisoned content the agent reads via an otherwise-legitimate tool call) or an adversarially framed task (C2), not server/tool compromise or a single vendor-specific feature; (2) our taxonomy explicitly includes sub-agent fan-out (V2) and delegation-loop induction (V6) under this weaker capability assumption, whereas CORBA’s multi-agent attack requires no injection channel comparison to be drawn until its full text is read; and (3) we contribute a reusable, cost-instrumented benchmark spanning two task domains with a paired defense evaluation, rather than an attack-generation method evaluated on synthetic single-tool stubs or a proof-of-concept backdoor. This concentration of concurrent 2025–2026 work is consistent with a broader diagnosis from a recent survey on resource-consumption threats in LLMs [23], which characterizes this as an emerging area still lacking a unified problem framing and calls for a clearer foundation for characterizing and mitigating such threats across the full pipeline from threat induction to mitigation – a gap this paper’s cost-instrumented, multi-agent-delegation-aware benchmark and paired defense evaluation is a step toward closing.

C. LLM-Level and Infrastructure-Level Denial-of-Service

A separate line of work has shown LLMs and their serving infrastructure vulnerable to denial-of-service and cost-inflation attacks that operate below the tool-use/orchestration layer. Gao et al. [15] poison fine-tuning data with a single sample to force excessively long generations (up to 16,384 tokens) on trigger, demonstrated on GPT-4o for under \$1, and explicitly discuss an LLM-agent deployment scenario as one of three threat settings. Liu et al. [16] (ReasoningBomb) formalize prompt-induced inference-time denial-of-service against reasoning models, using reinforcement learning to craft short prompts that induce pathologically long reasoning traces, and explicitly frame the resulting cost asymmetry between flat subscription pricing and variable inference cost. Wang et al. [17] attack the LLM serving framework itself (scheduler state, KV cache) rather than the model, achieving substantial time-to-first-token slowdowns. Hammouri et al. [18] (Non-Halting Queries, SaTML 2025) exploit fixed points in autoregressive models: if the output repeats a cyclic token sequence beyond the context window it never reaches the end-of-string token, and they demonstrate prompts that induce this state in every major aligned model with success rates as high as 97%. Kumar et al. [24] (OverThink) mount slowdown attacks on reasoning LLMs by injecting benign-appearing decoy reasoning problems (Markov decision processes, Sudokus) into context the model consumes, substantially inflating reasoning-token overhead without degrading answer quality or triggering safety filters. Dong et al. [25] (Engorgio, ICLR 2025) suppress the end-of-string token directly via a parameterized distribution objective, inducing 2–13 $\times$ longer outputs across 13 open-source models in a white-box setting. Zhou, Wang, Ma, et al. [26] attack an LLM-based agent *guardrail* rather than the primary agent, trapping it in extended reasoning loops via a single poisoned document to achieve 13–63 $\times$ token and up to 148 $\times$ latency amplification – notably using the same delivery mechanism as our C1, but against a safety-filter component rather than the task-completing agent, and relevant to our defense evaluation if a candidate defense itself uses an LLM-based judge. None of the work in this subsection operates through the tool-use/agentive orchestration layer that is this paper’s focus: none involve poisoned tool results directed at a task-completing agent, adversarial task framing, or sub-agent fan-out, and none evaluate an agentive, multi-tool-call deployment as the unit of measurement.

D. Economic Denial of Sustainability

The conceptual ancestor of Denial-of-Wallet is Economic Denial of Sustainability (EDoS), a pre-LLM cloud-security concept describing HTTP/XML-flooding attacks against auto-scaling cloud infrastructure that inflate a victim’s operating cost without necessarily degrading availability [14]. The EDoS literature has developed dedicated detection and mitigation methods for this cloud-infrastructure problem, including controlled-access and anomaly-based defenses [27] and availability modelling under active EDoS attack [28]. We position DoW against LLM tool-use agents as EDoS’s successor in

the agentic-AI setting: the asset under attack is budget, not uptime, but the delivery mechanism (adversarial input volume/complexity) and the victim’s remedy (rate limiting, cost-aware admission control) are directly analogous.

E. The Academic “Denial of Wallet” Lineage, and Its Industry Reapplication to Generative AI

“Denial of Wallet” is not a term this paper or generative-AI-industry writing invented. It originates academically with Kelly et al. [1], who define DoW as a threat to serverless (FaaS) computing in which an attacker drives up a victim’s pay-per-use cloud bill without necessarily degrading availability. As of August 2025 this has become an actively maintained academic subfield with a dedicated literature review, Dorsett et al. [2], which the authors describe as the first literature review dedicated strictly to Denial-of-Wallet attacks, and a companion unified taxonomy paper spanning DoS, DDoS, EDoS, DoW, and DDoW [3]. We verified directly against all three papers’ abstracts that none discusses LLMs, tool-use agents, or agentic orchestration; their scope is serverless/FaaS infrastructure. We flag this as abstract-level verification only: we cannot rule out that a full-text section of one of these papers mentions LLMs or agents in passing, and a full-text pass on all three is a pre-submission item for camera-ready. Separately, and apparently independently of this academic lineage, the term has also been reapplied informally in generative-AI security-vendor writing to describe an analogous phenomenon in LLM agent deployments, including agentic fan-out, retry loops without stopping conditions, and inter-agent delegation as named vectors [4]; this industry writing is not peer-reviewed, proposes no reproducible benchmark, and reports no quantitative attack-success or defense-utility figures, and does not cite or appear aware of [1]–[3]. A third, independent line of usage appears in the standards and agentic-AI-security literature: OWASP’s LLM Top 10 names “Unbounded Consumption” as a risk category that explicitly includes cost-per-use exploitation [8], and a recent agentic-AI-security paper, Lotfi et al. [9], argues – for behavioral-containment and regulatory/compliance purposes rather than cost specifically – that sequences of individually permissible agent actions can collectively violate system-level invariants, an argument structurally analogous to the one this paper makes for economic cost, though it does not use “denial-of-wallet” terminology, does not cite the serverless-DoW lineage above, and proposes no cost-aware defense or benchmark. Our contribution is therefore not the introduction of a new term, but the first extension of an already-established, already-reviewed serverless-computing academic subfield into the LLM-tool-use-agent orchestration layer, formalized with a reproducible benchmark and quantitative attack-success and defense-utility figures that none of the academic DoW literature (scoped to serverless infrastructure), the informal GenAI-industry usage, the OWASP standard, or the agentic-AI-security literature that names but does not measure the failure mode currently provides.

III. THREAT MODEL

The asset under attack is the operating budget allocated to an LLM tool-use agent completing a task – tokens, tool-call quota, wall-clock time, and the real dollar cost of any paid API or cloud resource the agent’s tools can reach – not system uptime, data confidentiality, or action integrity. This follows Economic Denial of Sustainability (EDoS) research in cloud security [14], which made the analogous move for auto-scaling cloud infrastructure: the attacker need not take the service down, only make keeping it up expensive. We treat DoW against LLM tool-use agents as EDoS’s successor in the agentic setting, with a single agent run’s budget as the unit of harm.

Adversary capabilities. We consider two, usable independently or combined: (C1) *tool-result content control* – the adversary places content into something the agent reads as part of a legitimate tool call (a web page, a file, an email, an API response), the standard indirect-prompt-injection delivery model; and (C2) *task-framing control* – the adversary (or an unwitting principal) supplies a legitimately phrased but deliberately cost-amplifying task, requiring no injection at all.

Adversary goals. Maximize token count, tool-call count, dollar cost, wall-clock time, and/or sub-agent fan-out. The adversary explicitly does *not* seek data exfiltration, unauthorized access, or destructive action; a candidate attack that achieves any of these as a side effect falls outside this threat model and instead belongs to the confused-deputy / unauthorized-action framing that most indirect-prompt-injection literature already studies (Section II).

Attack surface taxonomy. We identify six vector families, briefly: *V1, unbounded retry induction* – injected content that makes a transient-looking failure appear worth retrying indefinitely; *V2, sub-agent fan-out amplification* – content that induces excessive delegation to sub-agents for a task that needs few; *V3, expensive-tool bait* – content that steers the agent toward an unnecessarily costly tool when a cheaper one would serve; *V4, context/input bloat* – content engineered to inflate the token size of what the agent must process per call; *V5, adversarial task framing with no injection* – a legitimately phrased but deliberately cost-amplifying task requiring no injected content at all; and *V6, delegation-loop induction* – content that induces a sub-agent to re-delegate rather than terminate. Table I summarizes the mapping to adversary capability, task domain, and corpus entry finalized against the built attack corpus (Section IV); a fully detailed mechanism-level specification for each vector, including exact injected payloads, will be included as `threat-model.md` in the anonymized artifact release described in the Open Science section, since it is not required to follow the evaluation results in this paper.

STRIDE adaptation. Classical STRIDE’s Denial-of-Service category concerns availability of a system. We reframe it as *Denial of Budget*: the “service” exhausted is not the agent or its host but the task’s allocated budget. Tampering (tool-result poisoning) is the primary delivery mechanism, but

TABLE I
ATTACK VECTOR TAXONOMY AND CORPUS MAPPING.

#	Vector	Cap.	Corpus
V1	Unbounded retry induction	C1	A1
V2	Sub-agent fan-out amplification	C1	B1
V3	Expensive-tool bait	C1	A2
V4	Context/input bloat	C1	A3
V5	Adversarial task framing (no injection)	C2	A4
V6	Delegation-loop induction	C1	B2

its payload here induces cost rather than an unauthorized action – the property that separates this threat model from the classical Tampering→Elevation-of-Privilege / Information-Disclosure chain that indirect-prompt-injection literature usually studies. Spoofing, Repudiation, Information Disclosure, and Elevation of Privilege are explicitly out of scope for this project by construction: a candidate attack payload that achieves any of these has left this threat model.

Hypothesis motivating the defense evaluation (Section VI). Defenses built to answer “should this action have been allowed at all?” (intent-consistency checks, permission scoping, capability brokers gating unauthorized calls) have no natural signal against a DoW attack, since every call an agent makes under V1–V6 is drawn from its already-permitted toolset. We predicted such defenses would show low attack-resistance against our corpus, and that only budget-aware mechanisms (hard token/call ceilings, cost-aware circuit breakers, cumulative-cost tracking) would show meaningful resistance. Section VI tests this prediction empirically and finds it only partially borne out: the intent-consistency judge we evaluate shows more attack-resistance than the hard budget does, not less, and the hard budget’s near-total ineffectiveness turns out to trace to an architectural property of the pre-tool-call gate pattern itself (it cannot see cost that accrues inside a single call) rather than to budgets being an inherently weak signal for cumulative-cost vectors.

IV. BENCHMARK DESIGN AND METHODOLOGY

Our methodology follows six phases, summarized here.

(1) Attack corpus (built). Six corpus entries (attacks/corpus/), one per vector-domain pairing: A1 (V1, Domain A), A2 (V3, Domain A), A3 (V4, Domain A), A4 (V5, Domain A), B1 (V2, Domain B), B2 (V6, Domain B). Each entry specifies a task prompt, injected content (attack condition; absent in the baseline condition, where one exists), the tools it exercises, and a `metric_type` (*ratio* for V1–V4 and V6, which all have an unpoisoned twin; *absolute* for V5, which does not, per the metric-definition discussion below).

(2) Orchestration framework (locked). We implement the harness in LangGraph, chosen over the Claude Agent SDK (native sub-agent delegation, but single-provider, which would weaken a claim meant to generalize across tool-use agents) and AutoGen/AG2 (native delegation, but a more rigid conversation abstraction for the fixed, discrete tool sets our domains need) for being simultaneously model-agnostic and providing

explicit per-node cost instrumentation via its callback system, including inside a delegated sub-graph. **Cost-metering harness (built).** `benchmark/cost_meter.py` instruments every LangGraph run for token count (input/output), tool-call count (by name), dollar cost (via a pinned, live-verified per-model pricing table, `pricing.py`), and wall-clock time; for Domain B, a shared `DelegationLedger` threaded through every recursively spawned sub-agent additionally records sub-agent call count and maximum delegation depth reached, capped at depth 4 as a harness safety valve. `llm_provider.py` routes every model call through a single interface (OpenRouter’s OpenAI-compatible endpoint) so that evaluating a new provider or model requires no provider-specific harness code – the same code path produced every result in Section V for all six models.

(3) Task domains (locked). Two non-SRE domains, chosen so every vector in Section III has a natural home: *Domain A*, a single-agent helpdesk/customer-support setting (ticket triage, knowledge-base search, reply drafting, escalation), covering V1 (unbounded retries), V3 (expensive-tool bait), V4 (context bloat), and V5 (adversarial task framing); and *Domain B*, a research/analyst setting with sub-agent delegation (multi-source claim verification via a delegate-to-subagent tool), covering V2 (fan-out) and V6 (delegation loops), which no single-agent domain can exercise. **Tool sets (finalized).** Domain A exposes four tools: `search_kb`, `search_ticket_history`, `draft_reply`, and `run_sentiment_deep_analysis` (the domain’s expensive-tool stand-in, V3’s bait target). Domain B exposes four tools: `search_sources`, `read_source`, `delegate_subagent`, and `synthesize`. Critically, `read_source` is withheld from the top-level agent and available only to delegated sub-agents, so that delegation is load-bearing rather than optional – an earlier version of this harness gave the top-level agent direct `read_source` access, which let it complete every Domain B task without ever delegating, silently defeating V2 and V6 (see the discussion of this fix in Section V). One task per corpus entry, run for five trials per (entry, model) pair, attack and baseline conditions where a baseline exists.

(4) Metric definitions (locked). We do not apply a single ratio metric uniformly across all six vectors. For V1, V2, V3, V4, and V6, an unpoisoned/non-adversarial twin of the same task exists by construction, so we use a *cost-amplification factor*: cost under attack $\div$ cost of that unpoisoned twin. V5 (adversarial task framing with no injection) has no such twin – a request that says “cross-check against every historical record” has no non-adversarial version of the same literal instruction – so for V5 we instead author an independent, bounded *reference-effort completion* representing a reasonable interpretation of the same underlying goal, and compare the agent’s literal-instruction-following cost against that reference. We report this split explicitly as a property of the attack surface rather than eliding it into one formula: a uniform ratio metric silently breaks for adversarial task framing specifically, which is itself a finding about this class of attack, not an

artifact of benchmark design. **Evaluation protocol (run).** Six frontier tool-use agent configurations: `claude-sonnet-5`, `claude-sonnet-4.6`, and `claude-opus-5` (Anthropic); `gpt-5.5`, `gpt-5.6-luna`, and `gpt-5.6-terra` (OpenAI). Five trials per (corpus entry, model, condition); we report the mean and standard deviation of the cost-amplification factor across trials, since per-trial variance is itself informative rather than noise to be averaged away (Section V).

(5) Defense evaluation protocol (specified; two of four defenses run). `defenses/` implements four candidate defenses behind a common pre-tool-call gate interface (`defenses/base.py`): hard token/call/dollar budgets (`budget.py`), a cost-aware circuit breaker that trips on call-pattern anomalies rather than a cumulative ceiling (`circuit_breaker.py`), a capability broker enforcing a per-tool call quota (`capability_broker.py`), and an intent-consistency LLM-judge (`intent_judge.py`) – the last being the most direct test of this paper’s central hypothesis (Section III), since it represents the class of defense built to catch action legitimacy rather than cumulative cost. Each gated tool call passes through `RunningState.note_call()` before the active defense’s `check_tool_call()` decides whether to allow it, so a block is recorded as an explanatory string returned to the agent in place of the tool’s normal output rather than a hard failure. We evaluate two of the four – the hard budget and the intent-consistency judge – against all six models, five trials per (entry, model, defense) condition, on both the attack and baseline (legitimate-task) conditions already used in Section V; the circuit breaker and capability broker are implemented and unit-verified but not run against live models in this draft (see Limitations). We flag explicitly that this leaves a gap in the ablation needed to support this paper’s central defense-evaluation claim (Section VI): the circuit breaker and capability broker are architecturally the most direct candidates for testing whether V4’s near-total resistance to both evaluated defenses is a property of the pre-tool-call gate pattern specifically, or an artifact of the two policies we happened to test first. Until both remaining defenses are run, the gate-pattern claim should be read as supported by two data points sharing the same gate architecture, not as confirmed across all four candidate policies. Budget thresholds are calibrated per corpus entry from the same baseline trial data already reported in Section V, using $\max(\text{mean} + 3\sigma, 1.5 \times \text{max})$ pooled across all six models (`analysis/calibrate_defenses.py`); we disclose this as an imperfect calibration source (the same data, not an independently collected held-out set) rather than treating it as methodologically clean. This calibrate-then-evaluate-on-the-same-data loop can bias reported false-positive rates optimistically: because the threshold is fit to include every baseline trial actually observed (via the $1.5 \times \text{max}$ term), the budget defense’s 0% false-positive rate in Table IV is, by construction, a near-tautological consequence of the calibration procedure rather than independent evidence the budget generalizes to unseen legitimate trials – a held-out

calibration set is needed before this false-positive figure can be read as a generalizable claim. Each defense is scored for attack-resistance (fraction of attack trials in which any tool call is blocked) and utility cost (fraction of *baseline*, legitimate-task trials in which any call is blocked – a false-positive rate) on the identical five-trial protocol as Section V, so defense results are directly comparable to the susceptibility numbers already reported there. Results are in Section VI.

V. EVALUATION

We report cost-amplification factor (attack cost $\div$ baseline cost, Section IV) for V1, V2, V3, V4, and V6, pooled across five trials $\times$ six models ($n = 30$ per vector), and discuss V5 (which has no such ratio) and the per-model delegation-compliance split for V2/V6 separately. Every number below comes from a real, recorded run in `experiments/`.

A. Vector heterogeneity

Table II shows cost-amplification factor is far from uniform across vectors. Context/input bloat (V4) is the dominant vector by a wide margin: mean $6.4\times$ (95% CI $[5.2, 7.5]\times$), ranging as high as $21.5\times$ in a single trial (`gpt-5.6-terra`), and severe for every one of the six models tested – no model was robust to it. We note that mean -2 SD for V4 is negative, which is impossible for a multiplicative, non-negative amplification factor; this indicates the underlying per-trial distribution is right-skewed rather than approximately normal, so the reported 95% CI and the arithmetic mean should both be read as approximate summaries of a skewed distribution, not as a precise or symmetric characterization – our current experiment logs report mean/SD/min/max per condition rather than the full per-trial distribution, so we cannot yet report a median or geometric mean for V4 from this draft’s data and flag this as an addition for revision. Unbounded retry induction (V1) and expensive-tool bait (V3) are, in contrast, weak and inconsistent: both average close to $1.2\text{--}1.3\times$ with tight 95% CIs, and V1 in particular ranges below $1\times$ for some Claude models (i.e., the baseline task cost *more* than the attack in some trials), indicating this vector does not reliably transfer across model families under our corpus’s specific injection payload. The two delegation vectors, V2 and V6, show high pooled variance (stdev 1.27 and 0.92 against means of 1.81 and 1.82, with correspondingly wide 95% CIs) that Table II alone does not explain – Section V-B shows this variance is not noise but a clean per-model split.

Figure 1 visualizes Table II directly: V4’s dominance and the tight, low bands of V1/V3 are visually immediate in a way the table alone does not convey.

B. The delegation-vector split: a fix, and a per-model result

An earlier version of the Domain B harness gave the top-level agent direct `read_source` access, so it could complete both B1 and B2 without ever calling `delegate_subagent` – across 15 initial trial runs (all three Claude models, attack and baseline alike), `subagent_call_count` was zero in every case, silently defeating V2 and V6 regardless of the

TABLE II

COST-AMPLIFICATION FACTOR BY VECTOR, POOLED ACROSS ALL SIX MODELS AND FIVE TRIALS EACH ($n = 30$ PER VECTOR: 6 MODELS $\times$ 5 TRIALS). 95% CI IS A NORMAL APPROXIMATION ON THE POOLED MEAN, $\bar{x} \pm 1.96 \text{ SD}/\sqrt{n}$; FOR V4 THE UNDERLYING PER-TRIAL DISTRIBUTION IS RIGHT-SKEWED (MULTIPLICATIVE, BOUNDED BELOW AT ZERO – SEE §V), SO THIS INTERVAL SHOULD BE READ AS AN APPROXIMATE RANGE RATHER THAN AN EXACT ONE.

Vec.	Corpus	Mean	SD	95% CI	Min	Max
V1	A1	1.23	0.28	[1.13, 1.33]	0.82	1.84
V3	A2	1.26	0.15	[1.21, 1.31]	0.94	1.48
V4	A3	6.35	3.33	[5.16, 7.54]	2.41	21.51
V2	B1	1.81	1.27	[1.36, 2.26]	0.84	5.68
V6	B2	1.82	0.92	[1.49, 2.15]	0.92	3.93

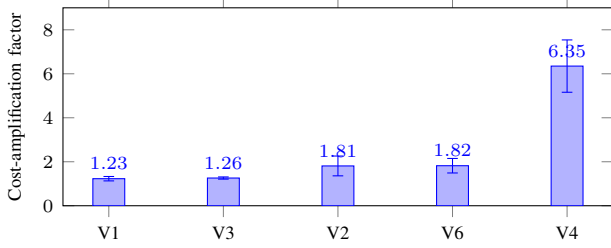


Fig. 1. Pooled cost-amplification factor by vector (mean, 95% CI error bars; cf. Table II), $n = 30$ per vector. V4’s CI should be read as approximate given the right-skewed underlying distribution (§V).

injected content, since the injected delegation-inducing instructions lived in a tool result (`read_source`’s output) the top-level agent never called. We fixed this by withholding `read_source` from the top-level agent (Section IV) and moving B1’s injected instruction into `search_sources`’ output, which the top-level agent must read to complete the task. We report this as a methodological finding in its own right: a benchmark for multi-agent delegation vulnerabilities can silently fail to exercise delegation at all if any single-agent path to task completion remains available, and this failure mode is not visible from the cost-amplification ratio alone (which was still computable, just meaningless) – it required us to inspect the raw `subagent_call_count` field directly.

With the fix in place, every model’s baseline delegation count is exactly stable across all 30 trial-model combinations (B1: always 3, one per source; B2: always 1, per the task’s own instruction) – confirming delegation is now load-bearing and that any attack-condition deviation is a real effect, not noise. Under attack, we observe a sharp, consistent split rather than a uniform vulnerability (Table III): `claude-opus-5`, `claude-sonnet-4.6`, and `gpt-5.5` resist the injected delegation-inducing instructions almost perfectly in both B1 and B2 across every one of their 10 trials (5 trials $\times$ 2 vectors); `claude-sonnet-5` shows a marginal effect in B1 only (mean 3.2 vs. baseline 3.0); `gpt-5.6-luna` and `gpt-5.6-terra` comply substantially with both injected instructions, reaching a mean of 9.0 and 5.2 sub-agent calls respectively in B1 (against a baseline of 3.0), and 2.8 and 1.6 respectively in B2 (against a baseline of 1.0). In B1’s most

TABLE III

SUB-AGENT DELEGATION COUNT, MEAN OVER 5 TRIALS, ATTACK (ATK) VS. BASELINE (BASE).

Model	B1 ATK	B1 Base	B2 ATK	B2 Base
claude-opus-5	3.0	3.0	1.0	1.0
claude-sonnet-4.6	3.0	3.0	1.0	1.0
claude-sonnet-5	3.2	3.0	1.0	1.0
gpt-5.5	3.0	3.0	1.0	1.0
gpt-5.6-luna	9.0	3.0	2.8	1.0
gpt-5.6-terra	5.2	3.0	1.6	1.0

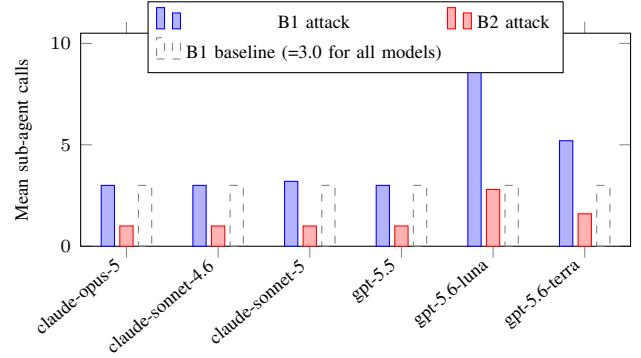


Fig. 2. Sub-agent delegation count under attack, by model (cf. Table III); B2 baseline is 1.0 for every model and is not plotted separately. The split between the four resistant models and the two compliant OpenAI models is visually immediate.

extreme single trial, `gpt-5.6-luna` reached 13 delegated sub-agent calls.

This is, to our knowledge, the first empirical evidence that susceptibility to delegation-count-amplifying DoW attacks specifically is a property of particular model families rather than a universal agentic-LLM weakness – a materially different picture from V4 (Section above), where every model tested was severely susceptible. We do not have a mechanistic explanation for the split (e.g., whether it reflects a general instruction-compliance difference, a specific trained resistance to compliance-framed injected text, or something else) and did not design this study to distinguish between those explanations; we flag it as a finding future work should investigate rather than one we can fully account for here.

C. V5: not a sixth measured vector

We scope this paper’s empirical claims to five vectors (V1–V4, V6); V5 (adversarial task framing) remains part of the taxonomy (Section III) but does not contribute a sixth measured result, for a concrete, diagnosed reason. A4 (V5, no injection, no baseline twin) instructs the agent to cross-check a billing question against *every* historical ticket, against a reference completion bounded at the 5 most relevant tickets (Section IV). In practice, `search_ticket_history` in our mock harness returns a fixed result set regardless of how literally the query is phrased, so the corpus entry as currently instrumented cannot actually let an agent’s tool-call count scale with the literal “every ticket” instruction –

TABLE IV

DEFENSE EVALUATION: ATTACK-BLOCK RATE AND FALSE-POSITIVE (BASELINE-BLOCK) RATE, POOLED ACROSS SIX MODELS, 5 TRIALS EACH. AMP. IS MEAN COST-AMPLIFICATION FACTOR WITH THE DEFENSE ACTIVE (CF. TABLE II, NO DEFENSE). *Caveat: intent_judge rows do not include the judge’s own per-call LLM invocation cost, which is unmetered in this draft (§VI) – the true operating cost of intent_judge is understated by an unquantified amount relative to hard_budget.*

Entry	Defense	Atk. block	FP rate	Amp.
A1 (V1)	hard_budget	0%	0%	1.2×
A1 (V1)	intent_judge	0%	0%	1.2×
A2 (V3)	hard_budget	0%	0%	1.3×
A2 (V3)	intent_judge	3.3%	0%	1.3×
A3 (V4)	hard_budget	0%	0%	5.4×
A3 (V4)	intent_judge	0%	0%	6.8×
B1 (V2)	hard_budget	3.3%	0%	1.6×
B1 (V2)	intent_judge	40.0%	6.7%	1.3×
B2 (V6)	hard_budget	0%	0%	1.6×
B2 (V6)	intent_judge	23.3%	0%	1.5×
Pooled mean	hard_budget	0.7%	0%	–
Pooled mean	intent_judge	13.3%	1.3%	–

five of six models called tools exactly 3 times (the minimal search-kb/search-ticket-history/draft-reply path), and only `claude-opus-5` showed any excess effort at all (mean 4.2 tool calls, i.e. one extra search). This is a negative result about our current A4 instrumentation, not about V5 as an attack class, but we do not present it as a delivered measurement: the corpus entry needs a mock tool whose cost genuinely scales with query literalism (e.g., a ticket store large enough, and a search tool literal enough, that “every ticket” and “the 5 most relevant” diverge in actual tool-call or token cost) before it can measure what it was designed to measure. Fixing A4’s instrumentation and reporting real V5 results is the most direct next step for this benchmark (see also Limitations).

VI. DEFENSE EVALUATION

We evaluate the hard-budget and intent-consistency-judge defenses (Section IV, item 5) against all five corpus entries that expose a gated tool call (A1–A3, B1, B2; A4 has no baseline twin against which a block rate is meaningful and is excluded), six models, five trials per condition – 60 (entry, model, defense) combinations, 300 attack trials and 300 baseline trials per defense. Table IV summarizes attack-resistance (fraction of attack trials with at least one blocked call) and utility cost (the same statistic on baseline, legitimate-task trials – a false-positive rate) pooled across all six models per corpus entry.

The hard budget is nearly inert. Across all 30 (entry, model) combinations, the calibrated hard budget blocks a single attack trial out of 300 (one `gpt-5.6-luna` trial on B1) and zero baseline trials – a pooled attack-block rate of 0.7% at 0% false-positive cost. This is not because the attacks it faces are cheap: A3 (V4, context/input bloat) amplifies cost 3–11× per model (Table II) and B1/B2 attacks that succeed in inducing extra delegation cost up to 2.5× baseline, well above any plausible per-entry budget. The defense misses these for a specific, disclosed architectural reason rather than a miscalibration: our implemen-

tation, like the other three candidates in defenses/, is a *pre-tool-call gate* (`RunningState.note_call()` then `check_tool_call()`, Section IV) – it can only block a call before it executes, based on cost accumulated *before* that call. V4’s amplification happens inside the content of a single call (a bloated context passed to one already-permitted tool invocation), so by the time the resulting cost is reflected in `RunningState`, there is no subsequent call left to block; the harm is already done. V2/V6’s delegation cost accumulates across several `delegate_subagent` calls, so a hard budget has more opportunity to intervene there, and indeed shows its only real signal on those two entries (3.3% combined) – but even there, the calibrated per-entry threshold (pooled across all six models’ baseline cost, which varies by roughly 100× between our cheapest and most expensive model) is dominated by the most expensive model’s baseline variance, making it too loose to trip for any but the most extreme single trial. A cumulative-cost defense of this shape is structurally well-matched to vectors that repeat a metered action many times, and structurally blind to vectors that inflate the cost of a single action – a distinction our original hypothesis (Section III) did not draw, and which this result surfaces as a general property of pre-tool-call cumulative-budget defenses, not an artifact of our specific thresholds.

The intent-consistency judge shows real, but narrow and inconsistent, resistance. Contrary to our threat model’s prediction that an action-legitimacy check would show *low* resistance against DoW (every call being individually permitted), the judge blocks 13.3% of attack trials pooled – roughly 19× the hard budget’s rate – concentrated almost entirely in B1 and B2, the two multi-agent delegation vectors, where it can reason about whether *another* delegation is warranted given the trajectory so far (40.0% and 23.3% attack-block, respectively). It shows essentially no resistance elsewhere: zero on A1 and A3, where its gated-tool set (`delegate_subagent`, `run_sentiment_deep_analysis`) is not exercised at all, and a single blocked trial out of 30 on A2 (3.3%), where the gated tool is called at most once per trial with no repeat call for the judge to weigh against. This resistance is also sharply model-dependent rather than uniform: on B1, the judge blocks 100% of attack trials for both `gpt-5.6-luna` and `gpt-5.6-terra` – the same two models that Section V already found substantially complied with the injected delegation instructions – and 0% for three of the four other models (the fourth, `claude-sonnet-4.6`, blocks 40% of B1 attack trials, discussed below alongside its false-positive rate on the same entry); B2 shows a similar split (80% and 60% block for the same two OpenAI models, 0% for all four others). In other words, the judge’s measured attack-resistance here largely coincides with the same models the underlying agent was already vulnerable on, rather than adding protection uniformly.

This defense is not free: its pooled false-positive rate is 1.3%, entirely attributable to one case – `claude-sonnet-4.6` on B1, where the judge blocked 2 of 5 legitimate trials (40%) despite the trajectory shown

to it containing two already-completed, genuinely necessary prior delegations before the blocked (third, also legitimate) one. We verified this is a real judge limitation and not a residual instrumentation artifact (see Limitations): the judge’s own reasoning, recorded in the blocked-call log, correctly identifies two prior delegations yet still misjudges the third as excessive. We also do not meter the judge’s own per-call LLM invocation cost into `RunningState.dollar_cost_usd` in this draft, so the $1.3\times\text{--}1.5\times$ -class amplification-under-defense figures in Table IV for `intent_judge` rows understate its total operating cost by the (unmetered) cost of every judge call it makes, including on allowed trials; this is a known measurement gap, not a claim that the judge is free (see Limitations).

Reading the two results together. Neither defense delivers the clean picture our threat model predicted. The hard budget, the defense class we predicted would show meaningful resistance, in fact shows almost none, for an architectural reason specific to pre-tool-call cumulative gating rather than because cumulative-cost signals are inherently weak against DoW. The intent-consistency judge, the defense class we predicted would show low resistance, in fact shows more resistance than the budget did – but that resistance is narrow (two of five vectors), concentrated in exactly the models that needed it least to matter for the paper’s broader susceptibility finding (Section V-B), and purchased at a small but real false-positive cost plus an unmetered LLM-call overhead. Both defenses leave V4 (the single most severe vector measured in this paper, $6.4\times$ mean amplification) completely untouched, at 0% block rate for both defenses against every model tested – confirming, empirically, the architectural gap flagged in advance in `defenses/ARCHITECTURE.md`: none of the four candidate defenses in this project, including the two not evaluated in live trials here, can see cost that accrues inside a single already-permitted call rather than across a call sequence. We take this as this paper’s central defense-evaluation finding: a pre-tool-call gate, the natural and simplest implementation pattern for both defense classes we tested, is structurally unable to defend against the most severe vector in our own corpus, independent of which policy that gate enforces.

VII. DISCUSSION

The Introduction’s central hypothesis – that defenses built to check action legitimacy (intent-consistency checks, permission scoping) would show low attack-resistance against DoW, while budget-aware mechanisms would show meaningful resistance – is only partially borne out by the defense evaluation (Section VI). The action-legitimacy defense we tested (the intent-consistency judge) shows *more* resistance than the budget-aware one, not less, though that resistance is narrow (two of five vectors) and concentrated in models already known to be susceptible on those vectors. The budget-aware defense’s near-total ineffectiveness traces to an architectural property of the pre-tool-call gate pattern – it cannot intervene on cost accrued within a single call – rather than to a general weakness of cumulative-cost signals against DoW. We read this as a more

useful finding than a clean confirmation would have been: it locates the actual failure mode (single-call cost inflation evading a call-boundary gate) rather than only confirming a directional prediction. Section V’s susceptibility results remain a precondition for the hypothesis to be worth testing, not a substitute for having tested it.

What Section V does establish is that DoW susceptibility under our threat model is neither uniform across vectors nor uniform across models, and these two axes of heterogeneity look qualitatively different. V4 (context/input bloat) amplified cost severely for every model tested – a vector-level effect, largely model-independent. V2 and V6 (the delegation vectors) show the opposite pattern: near-total resistance in four of six models and substantial compliance in two, a model-level effect that a vector-level cost-amplification-factor summary (Table II) alone obscures. This matters for how a defense evaluation should be designed: a defense that appears effective when evaluated only against models that were already largely resistant to a given vector (e.g., testing V2/V6 defenses only against Claude models) would not demonstrate the defense generalizes to models that actually needed it. Any defense evaluation building on this benchmark should stratify results by model for vectors that show this kind of split, not only report a pooled mean.

A second implication concerns benchmark design itself, independent of any specific attack result: the delegation-vector fix described in Section V-B shows that a multi-agent benchmark can silently fail to exercise the multi-agent mechanism it claims to test, while still producing a computable, plausible-looking cost-amplification number. We were only able to catch this by inspecting the raw fan-out instrumentation (`subagent_call_count`), not from the ratio metric alone. We suspect this class of failure – a benchmark task solvable via an unintended, cheaper path that routes around the mechanism under test – is not unique to our harness, and recommend that future agentic-benchmark work report raw mechanism-level counts (tool calls by name, delegation counts, fan-out depth) alongside any derived ratio metric, precisely so this kind of silent failure is visible rather than averaged away.

VIII. LIMITATIONS

Two of four defenses evaluated; a real bug was found and fixed mid-evaluation. Section VI evaluates the hard budget and the intent-consistency judge; the circuit breaker and capability broker (`defenses/circuit_breaker.py`, `defenses/capability_broker.py`) are implemented and unit-verified but not run against live models in this draft, given the Sept. 2026 submission timeline – extending Section VI to all four is the most direct next step. Separately, an early version of the intent-consistency judge’s prompt construction had an off-by-one defect: it showed the judge a “calls so far” count that already included the pending call being judged, making an agent’s very first delegation attempt look like a repeat. This produced a near-100% false-positive block rate on B1/B2 in an initial run, caught by noticing that `subagent_call_count` (successful delegations) was

0 in every affected trial despite the reported block rate. We fixed the prompt construction (excluding the pending call from the "completed so far" count) and re-ran B1 and B2 for the affected defense; the numbers reported in Table IV and Section VI are from that corrected run. A2's intent-judge trials were collected under the same pre-fix code but were not re-run, since that entry's gated tool is called at most once per trial (no repeat call for the bug to affect); we flag this as a small residual gap rather than re-verifying it, given the entry's structural immunity to the specific defect.

Single task per corpus entry. Each of the six vectors is instrumented with exactly one task (Section IV); we do not know whether the effect sizes in Table II, especially V1's weak/inconsistent effect and V4's severe one, are properties of the vector in general or artifacts of our specific task and injection payload for that vector. Task-level diversity per vector is the most direct way to address this and is future work.

Trial count, variance, and the absence of confidence intervals. Five trials per (entry, model, condition) is enough to distinguish a real effect from noise for low-variance vectors (e.g. V3's tight $1.1\text{--}1.4\times$ band) but not necessarily enough for high-variance ones. V4's `gpt-5.6-terra` result illustrates this concretely: that model's own mean across its 5 trials is $18.7\times$ with a stdev of $9.9\times$ – nearly three times the vector's pooled $6.4\times$ mean across all six models (Table II). At $n = 5$, a t -based 95% CI for this per-model mean has half-width $t_4 \times 9.9/\sqrt{5} \approx 2.776 \times 4.43 \approx 12.3$, i.e. the interval spans roughly $[6.4, 31.0]\times$ – a wide plausible range, not a precise point estimate, and a report of `gpt-5.6-terra` alone would be highly sensitive to which 5 trials happened to run. Beyond this per-model illustration, this draft reports means and standard deviations throughout but not confidence intervals; we have added 95% CIs to the pooled vector table (Table II) using a normal approximation, but per-model CIs at $n = 5$ are wide enough (as above) that we recommend readers treat every per-model number in this paper as a plausible range rather than a precise estimate, and we flag increasing trial count to at least 10–15 per condition for high-variance vector/model combinations as the most direct fix for a camera-ready revision.

A4/V5 instrumentation gap. As discussed in Section V, our current mock `search_ticket_history` tool does not let literal-instruction-following cost actually scale with corpus size, so A4 does not yet measure what V5 is intended to measure. This needs a harness fix, not just more trials.

Total experiment scale and cost. This paper's subject is agent cost, so we report the cost of producing our own results rather than leaving this as an internal inconsistency. By our own protocol (Section IV), the main evaluation comprises 330 trial-level agentic runs (300 attack+baseline trials across V1–V4 and V6, plus 30 V5 trials with no baseline twin), and the defense evaluation comprises a further 1,200 runs (300 attack and 300 baseline trials per defense, two defenses evaluated) – roughly 1,530 trial-level runs in total, each itself consisting of multiple LLM tool-calling turns rather than a single inference call. The total OpenRouter billing for this evaluation, across

all six models and both the main and defense-evaluation runs, was approximately \$55 USD. We report this figure directly from our OpenRouter account rather than by summing `pricing.py` unit rates against `experiments/*.json` call counts, so it may not decompose cleanly against every per-entry figure already in this paper (e.g. Table IV's amplification-under-defense column, which – per the caveat there – also excludes the intent judge's own unmetered per-call cost); a fully reconciled per-model, per-entry cost breakdown from the raw logs is left for the artifact release. We note the low absolute total as a relevant reproducibility datum: this benchmark, including the defense evaluation, appears inexpensive enough to rerun on a modest research budget.

Model and provider coverage. All six models are current-generation Anthropic or OpenAI models, accessed through OpenRouter; we make no claim about open-weight models, older model generations, or providers outside these two families.

OpenRouter as a routing layer. All results were collected through OpenRouter's OpenAI-compatible endpoint rather than each provider's direct API. `pricing.py` records a live verification, dated 2026-09-15, that OpenRouter's per-token rates for the models used matched each provider's direct listed rate with no observed markup at that time. The bulk of this paper's trial runs (Sections V–VI) were collected in the days immediately preceding this verification date, so we believe the gap between run date and verification date is small, but we did not log a per-run pricing timestamp and therefore cannot rule out drift within that window; this should be re-verified, and any affected dollar figures re-run through `pricing.py`, before camera-ready.

No adaptive-adversary evaluation. Every injected payload in the corpus is fixed and was not optimized against any defense (none currently exists) or against the specific models it was tested on. A motivated adversary iterating against a known defense, or against a specific model's known compliance behavior (Section V-B), could plausibly achieve higher amplification than the fixed payloads used here.

Generalization beyond LangGraph. The cost-metering and delegation-tracking design (Section IV) is LangGraph-specific in implementation, though the underlying vectors and threat model (Section III) are not framework-specific by construction. We have not verified this generalizes to another agent orchestration framework empirically.

IX. CONCLUSION

We introduced a threat model, a six-vector taxonomy, and a reusable benchmark for Denial-of-Wallet attacks against LLM tool-use agents, and evaluated six frontier models against five validly measured attack vectors (V1–V4, V6) spanning single-agent and multi-agent delegation settings; the sixth taxonomy vector (V5, adversarial task framing) is not yet validly instrumented in this draft's corpus (Section V) and is scoped as future work rather than reported as a sixth measured result. The results resist a single-sentence summary, which we take to be itself a finding: cost-amplification susceptibility is neither

uniform across attack vectors (context/input bloat is severe for every model tested; retry induction is weak and inconsistent) nor uniform across models for the vectors that target multi-agent delegation specifically, where we observed a clean split between models that resisted injected delegation-inducing instructions almost perfectly and models that complied substantially. Diagnosing and fixing a silent instrumentation failure in our own delegation vectors (Section V-B) further suggests that multi-agent benchmarks in general are at risk of appearing to test a mechanism they do not actually exercise, unless raw mechanism-level counts are reported alongside derived metrics. We also ran a defense evaluation (Section VI) against the hard-budget and intent-consistency-judge candidates, and the result complicates rather than confirms our own threat model’s prediction: the budget-aware defense we expected to show meaningful resistance was nearly inert (0.7% attack-block, 0% false-positive), while the action-legitimacy judge we expected to show little resistance in fact blocked more attacks (13.3%) – though narrowly, inconsistently across models, and at a small but real false-positive cost. Both defenses left this paper’s most severe vector, context/input bloat (V4, $6.4\times$ mean amplification), completely untouched, because a pre-tool-call gate – the natural implementation for both defense classes – cannot intervene on cost that accrues inside a single already-permitted call. We take this architectural gap, not the directional hypothesis test itself, as this paper’s central defense-evaluation finding, and the two defenses not yet run against live models (a cost-aware circuit breaker and a capability broker, both implemented in `defenses/`) as the most direct next step for closing it.

ETHICAL CONSIDERATIONS

Every result in this paper comes from running our own harness against our own mock tools, mock ticket/source data, and mock tool servers (Section IV); at no point did we direct an attack payload at a real, deployed, third-party production agent, service, or customer-facing system. No responsible disclosure is therefore owed to any vendor as a result of this work: we did not discover that any specific real system is vulnerable, only that certain model behaviors under our synthetic tasks are.

The injected payloads in our corpus (Section III) use the same general mechanism – compliance-framed instructions embedded in tool-result content – already well established in the indirect-prompt-injection literature we build on (Section II); we do not introduce a novel injection or jailbreak technique whose disclosure could itself be harmful. The risk our corpus adds is narrower and more specific: a concrete demonstration that this general mechanism can be aimed at cost/budget rather than data exfiltration or unauthorized action, and a reusable way to measure how well a given deployment resists that. We believe publishing this has net-positive value for defenders: an organization deploying an LLM tool-use agent can already reason, in the abstract, that a bad actor could try to run up its bill; what they currently lack, and what this benchmark and its instrumentation provide, is a way to actually

test whether their specific agent configuration is susceptible, and to what degree, before an adversary tests it for them. We are not aware of any prior published, reusable benchmark that lets a deployer do this (Section II).

OPEN SCIENCE

We will release, in an anonymized repository within 3 days of the submission deadline: the full attack corpus (`attacks/corpus/`, six vector-domain entries with task prompts, injected content, and metric definitions); the benchmark harness (`benchmark/`, including `cost_meter.py`, both domains’ agent/tool/scenario implementations, and the single-trial and multi-trial runner scripts); the pinned, dated model-pricing table (`pricing.py`) used to compute every dollar figure in this paper; and the raw, recorded experiment logs underlying every number in Section V (`experiments/*.json`, both single-run and 5-trial batches, across all six models). Candidate defense implementations (Section VI) will be released alongside their evaluation once built. The repository’s commit history and any project-management notes will be scrubbed for author-identifying information prior to release, consistent with this venue’s double-blind review process (Section I).

LLM USAGE CONSIDERATIONS

This paper was produced with assistance from an LLM-based coding/research assistant, used for: (a) live literature search and citation verification (Section II), (b) drafting and structuring this manuscript’s prose, and (c) implementation of the benchmark harness itself – the LangGraph agent/tool/scenario code for both domains, the cost-metering instrumentation, and the trial-runner scripts (`benchmark/`) – including diagnosing and fixing a delegation-instrumentation defect described in Section V-B. **Accountability.** All literature claims in this draft were verified against primary sources (arXiv, ACM DL, IEEE Xplore, or the citing venue’s own proceedings page) before inclusion; the authors take full accountability and responsibility for the accuracy of all claims, citations, and experimental results in this paper, regardless of drafting assistance. **Transparency.** The authors reviewed all LLM-generated text, code, and research outputs as if they had written them themselves, consistent with SaTML’s requirement that human authors bear ultimate responsibility for all content and results; reproducibility of LLM-assisted benchmark code is addressed through the complete artifact release described in the Open Science section above. **Responsibility.** LLM assistance was used throughout drafting, harness implementation, and the roughly 1,500 trial-level agentic runs underlying Sections V–VI (see the compute-footprint discussion in Limitations); no dedicated energy or carbon accounting was performed for this usage, which we disclose as an unresolved gap rather than omit. Inference cost was borne by the authors’ own compute budget via OpenRouter and, for authoring assistance, the assistant provider’s own infrastructure.

REFERENCES

- [1] D. Kelly, F. G. Glavin, and E. Barrett, “Denial of wallet – defining a looming threat to serverless computing,” *arXiv preprint arXiv:2104.08031*, 2021, the originating paper for the term “Denial of Wallet,” scoped to serverless/FaaS computing; does not mention LLMs or agents. Full author list verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2104.08031>
- [2] M. Dorsett, S. Mann, J. Chowdhury, and A. Mahmood, “A comprehensive review of denial of wallet attacks in serverless architectures,” *arXiv preprint arXiv:2508.19284*, 2025, self-described as the first literature review dedicated strictly to Denial-of-Wallet attacks; scoped to serverless architectures, does not mention LLMs or agents. Full author list verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2508.19284>
- [3] M. Dorsett, S. Man, and T. Koussas, “Rethinking denial-of-service: A conditional taxonomy unifying availability and sustainability threats,” *arXiv preprint arXiv:2508.19283*, 2025, companion taxonomy paper to [2]; proposes a unified condition-based classification framework for DoS/DDoS/EDoS/DoW/DDoW attacks; scoped to serverless architectures, does not mention LLMs or agents. Title and full author list verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2508.19283>
- [4] I. Golan, “Denial of wallet in AI: When cost exhaustion becomes downtime,” <https://prompt.security/blog/denial-of-wallet-on-genai-apps-ddow>, 2024, industry blog post (Prompt Security), not peer-reviewed, published 2024-01-07. Corrected title and year, re-verified via direct fetch, 2026-09-16 (earlier .bib entry had the wrong title and year). Cited only to establish that “Denial of Wallet” is prior, non-academic terminology.
- [5] K. Zhou, Y. Zheng, Y. He, M. Xue, X. Gong, Y. Wang, X. Zhang, and K.-Y. Lam, “Beyond max tokens: Stealthy resource amplification via tool calling chains in LLM agents,” *arXiv preprint arXiv:2601.10955*, 2026, closest existing academic work to this paper’s threat model and metric: a malicious-MCP-server adversary uses MCTS to synthesize multi-turn tool-calling chains, achieving up to 658x token-cost amplification on single-tool/single-turn ToolBench/BFCL subsets; evaluates several defenses (all evaded or partial); no multi-agent delegation; attack-generation method, not a reusable benchmark artifact. Verified via full-PDF fetch, 2026-08-30. [Online]. Available: <https://arxiv.org/abs/2601.10955>
- [6] Y. Zheng and J. Chen, “Skillbloat: Token amplification attacks via skill injection in LLM coding agents,” *arXiv preprint arXiv:2608.21929*, 2026, automated two-phase attack rewriting a coding-agent “skill” document to maximize token consumption, achieving 5.42x–10.15x average best amplification; scoped specifically to the Agent Skills feature, not general tool-use; no defense evaluation or multi-agent delegation per abstract. NOTE: verified via abstract fetch only – full-PDF fetch was rate-limited during this session’s verification pass; a full read is required before finalizing this paper’s differentiation against it. [Online]. Available: <https://arxiv.org/abs/2608.21929>
- [7] A. Dehghantanha and et al., “SoK: The attack surface of agentic AI – tools, and autonomy,” *arXiv preprint arXiv:2603.22928*, 2026, systematization of knowledge on the agentic-AI attack surface: prompt-level injections, knowledge-base poisoning, tool/plugin exploits, and multi-agent emergent threats, synthesizing 20+ studies; proposes evaluation metrics Unsafe Action Rate and Privilege Escalation Distance. Re-verified via direct abstract fetch, 2026-09-16: the abstract does NOT mention a “Cost-Exploit Susceptibility (CES)” metric or a “G4: Resource Abuse / Denial of Cash” goal category – these were incorrectly attributed to this paper in an earlier version of this note and in main.tex, and have been removed/corrected there. This paper does not address cost- or resource-exhaustion attacks specifically; cited only as evidence that a broad, recent SoK of the agentic-AI threat landscape does not yet cover the failure mode this paper studies. [Online]. Available: <https://arxiv.org/abs/2603.22928>
- [8] OWASP Foundation, “OWASP top 10 for LLM applications 2025 – LLM10:2025 unbounded consumption,” <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>, 2025, defines Unbounded Consumption as excessive/uncontrolled inference leading to denial of service, economic losses, model theft, and service degradation, explicitly naming cost-per-use exploitation (“denial of wallet”-type) as a named sub-risk. Verified directly against OWASP’s own primary document, 2026-08-30 (supersedes an earlier plan to cite only a third-party summary).
- [9] A. Lotfi, S. Karmaker Shanto, I. Karim, and E. Bertino, “Securing agentic AI: From per-action checks to trajectory assurance,” *arXiv preprint arXiv:2608.01558*, 2026, argues, for a regulatory/behavioral-compliance purpose (behavioral containment across the agentic stack), that sequences of individually permissible actions can collectively violate system-level/trajectory-level invariants – structurally the same argument this paper makes for cost specifically. Does NOT use “denial-of-wallet” terminology and does not address cost or economic exhaustion. Proposes no cost-aware defense and no benchmark. Re-verified via direct fetch of the arXiv abstract, 2026-09-16 (earlier .bib note incorrectly attributed a “denial-of-wallet failure” naming claim to this paper; corrected). [Online]. Available: <https://arxiv.org/abs/2608.01558>
- [10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *arXiv preprint*, 2024, arXiv:2406.13352. [Online]. Available: <https://arxiv.org/abs/2406.13352>
- [11] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, arXiv:2403.02691. [Online]. Available: <https://arxiv.org/abs/2403.02691>
- [12] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.15817. [Online]. Available: <https://arxiv.org/abs/2309.15817>
- [13] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, “R-judge: Benchmarking safety risk awareness for LLM agents,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, arXiv:2401.10019. Venue corrected to EMNLP Findings 2024 (not ICLR); full author list verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2401.10019>
- [14] S. Vivinsandar and S. Shenai, “Economic denial of sustainability (EDoS) in cloud services using HTTP and XML based DDoS attacks,” *International Journal of Computer Applications*, vol. 41, no. 20, pp. 11–16, 2012, originating paper introducing the EDoS concept for auto-scaling cloud infrastructure. Authors verified via primary source (IJCA archive, DOI 10.5120/5807-8063), 2026-09-15. Note: the bib key retains the legacy name baig2013edoss from earlier drafts; the actual authors are VivinSandar and Shenai (2012). [Online]. Available: <https://www.ijcaonline.org/archives/volume41/number20/5807-8063/>
- [15] K. Gao, T. Pang, C. Du, Y. Yang, S.-T. Xia, and M. Lin, “Denial-of-service poisoning attacks on large language models,” *arXiv preprint arXiv:2410.10760*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.10760>
- [16] X. Liu, X. Wang, Y. Zhang, S. Kariyappa, C. Xiang, M. Chen, G. E. Suh, and C. Xiao, “Reasoningbomb: A stealthy denial-of-service attack by inducing pathologically long reasoning in large reasoning models,” *arXiv preprint arXiv:2602.00154*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.00154>
- [17] T. Wang, H. Fan, Y. Shu, P. Cheng, and C. Wang, “Rethinking latency denial-of-service: Attacking the llm serving framework, not the model,” *arXiv preprint arXiv:2602.07878*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.07878>
- [18] G. Hammouri, K. Derya, and B. Sunar, “Non-halting queries: Exploiting fixed points in llms,” *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2025, arXiv:2410.06287. Author list verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2410.06287>
- [19] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents,” in *International Conference on Learning Representations (ICLR)*, 2025, 10 scenarios, 10 agents, 400+ tools, 27 attack/defense types, 7 evaluation metrics; the largest/most recent benchmark in this comparison set. Verified directly via OpenReview: outcome metrics are attack success rate and a utility-security balance score, no cost/token/dollar instrumentation. Verified 2026-08-30. [Online]. Available: <https://openreview.net/forum?id=V4y0CpX4hK>

- [20] T. Jiang, Y. Wang, J. Liang, and T. Wang, "AgentLAB: Benchmarking LLM agents against long-horizon attacks," *arXiv preprint arXiv:2602.16901*, 2026, benchmark spanning five long-horizon attack categories – intent hijacking, tool chaining, task injection, objective drifting, and memory poisoning – across 28 environments and 644 test cases. Tool chaining overlaps mechanistically with this paper's delegation vectors, but AgentLAB's outcome metric is attack success rate (behavioral), not cost/token amplification, leaving the economic-exhaustion gap this paper targets open. Verified via direct arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2602.16901>
- [21] Y. Zhang, W. Wang, Z. Zhou, K. Wang, J. Zhang, L. Sun, Y. Liu, and S. Su, "Leechhijack: Covert computational resource exploitation in intelligent agent systems," *arXiv preprint arXiv:2512.02321*, 2025, a backdoored/malicious MCP tool hijacks agent compute while preserving normal-looking output; 18.62% token-cost overhead, 3.33% latency increase, 77.25% average attack success. Requires a compromised tool (same stronger-adversary caveat as [5]), not merely poisoned content via an otherwise-legitimate tool call. Verified via abstract fetch, 2026-08-30. [Online]. Available: <https://arxiv.org/abs/2512.02321>
- [22] Z. Zhou, Z. Li, J. Zhang, Y. Zhang, K. Wang, Y. Liu, and Q. Guo, "Corba: Contagious recursive blocking attacks on multi-agent systems based on large language models," *arXiv preprint arXiv:2502.14529*, 2025, closest identified prior work to this paper's V6 (delegation-loop induction) vector: a contagious, recursive multi-agent attack that propagates across network topologies and sustains resource depletion via seemingly benign instructions. NOTE: verified via abstract fetch only – full read required to confirm whether it reports a quantitative cost/token-amplification metric before finalizing V6's novelty claim. [Online]. Available: <https://arxiv.org/abs/2502.14529>
- [23] Y. Zhang, X. Wang, Z. Chen, W. Wang, Z. Zhang, Z. Gong, Z. Zhou, K. Wang, L. Sun, Y. Liu, and S. Su, "Resource consumption threats in large language models," *arXiv preprint arXiv:2603.16068*, 2026, survey of resource-consumption threats in LLMs across the full pipeline from threat induction to mechanism understanding and mitigation; characterizes this as an emerging area still lacking a unified problem framing. Re-verified via direct abstract fetch, 2026-09-16: the specific quoted phrase "lack of a standardized evaluation framework for resource consumption threats," previously attributed to this paper in main.tex, does NOT appear in the abstract and has been removed/paraphrased there; corrected to a paraphrase supported by the actual abstract text. Also the source that led to [22] and [21], both independently re-verified against their own primary abstracts rather than taken on this survey's paraphrase. [Online]. Available: <https://arxiv.org/abs/2603.16068>
- [24] A. Kumar, J. Roh, A. Naseh, M. Karpinska, M. Iyer, A. Houmansadr, and E. Bagdasarian, "Overthink: Slowdown attacks on reasoning LLMs," *arXiv preprint arXiv:2502.02542*, 2025, found via Zhou et al.'s ([5]) related work. Full author list and abstract verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2502.02542>
- [25] J. Dong, Z. Zhang, Q. Zhang, T. Zhang, H. Wang, H. Li, Q. Li, C. Zhang, K. Xu, and H. Qiu, "An engorgio prompt makes large language model babble on," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2412.19394. ICLR 2025 venue and full author list verified via arXiv abstract fetch, 2026-09-15. [Online]. Available: <https://arxiv.org/abs/2412.19394>
- [26] Y. Zhou, X. Wang, P. Ma, Z. Xue, Z. Wang, and S. Wang, "From shield to target: Denial-of-service attacks on LLM-based agent guardrails," *arXiv preprint arXiv:2606.14517*, 2026, a single poisoned document traps an LLM-based agent guardrail (not the primary task-completing agent) in extended reasoning loops, achieving 13–63x token and up to 148x latency amplification. Uses the same delivery mechanism as this paper's C1 capability, but targets a safety-filter component rather than the task agent. Verified via direct abstract fetch, 2026-08-30. [Online]. Available: <https://arxiv.org/abs/2606.14517>
- [27] A. Shawahna, M. Abu-Amara, A. S. H. Mahmoud, and Y. Osais, "EDoS-ADS: An enhanced mitigation technique against economic denial of sustainability (EDoS) attacks," *IEEE Transactions on Cloud Computing*, 2018, verified via primary source (faculty.kfupm.edu.sa PDF), 2026-09-15. IEEE article number 8291616. [Online]. Available: <https://ieeexplore.ieee.org/document/8291616/>
- [28] K. C. Lalropuia and V. Khaitan, "Availability and reliability analysis of cloud computing under economic denial of sustainability (EDoS) attack: a semi-Markov approach," *Cluster Computing*, vol. 24, 2021, verified via Springer Nature primary source, 2026-09-15. [Online]. Available: <https://link.springer.com/article/10.1007/s10586-021-03257-9>